



BIS Working Papers

No 1268

Towards verifiability of total value locked (TVL) in decentralized finance

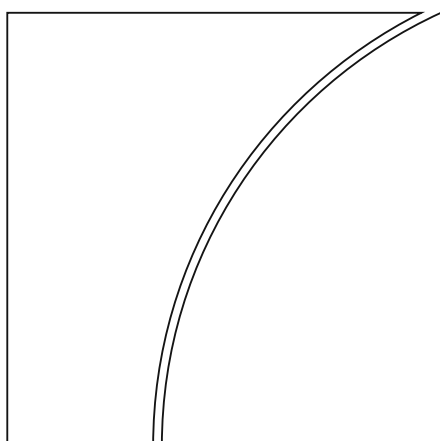
by Pietro Saggese, Michael Fröwis, Stefan Kitzler, Bernhard Haslhofer and Raphael Auer

Monetary and Economic Department

May 2025

JEL classification: E42, E58, F31, G12, G19, G23, L50, O33.

Keywords: Decentralized finance, DeFi, total value locked, TVL, Ethereum.



BIS Working Papers are written by members of the Monetary and Economic Department of the Bank for International Settlements, and from time to time by other economists, and are published by the Bank. The papers are on subjects of topical interest and are technical in character. The views expressed in them are those of their authors and not necessarily the views of the BIS.

This publication is available on the BIS website (www.bis.org).

© *Bank for International Settlements 2025. All rights reserved. Brief excerpts may be reproduced or translated provided the source is stated.*

ISSN 1020-0959 (print)
ISSN 1682-7678 (online)

Towards Verifiability of Total Value Locked (TVL) in Decentralized Finance

1st Pietro Saggese
IMT School for Advanced Studies Lucca
Lucca, Italy
pietro.saggese@imtlucca.it

2nd Michael Fröwis
Iknaio Cryptoasset Analytics
Vienna, Austria
michael@ikna.io

3rd Stefan Kitzler
Complexity Science Hub
Vienna, Austria
kitzler@csh.ac.at

4th Bernhard Haslhofer
Complexity Science Hub
Vienna, Austria
haslhofer@csh.ac.at

5th Raphael Auer
Bank for International Settlements
Basel, Switzerland
raphael.auer@bisih.org

Abstract—Total Value Locked (TVL) aims to measure the aggregate value of cryptoassets deposited in Decentralized Finance (DeFi) protocols. Although blockchain data is public, the way TVL is computed is not well understood. In practice, its calculation on major TVL aggregators relies on self-reports from community members and lacks standardization, making it difficult to verify published figures independently. We thus conduct a systematic study on 939 DeFi projects deployed in Ethereum. We study the methodologies used to compute TVL, examine factors hindering verifiability, and ultimately propose standardization attempts in the field. We find that 10.5% of the protocols rely on external servers; 68 methods alternative to standard balance queries exist, although their use decreased over time; and 240 equal balance queries are repeated on multiple protocols. These findings indicate limits to verifiability and transparency. We thus introduce “verifiable Total Value Locked” (vTVL), a metric measuring the TVL that can be verified relying solely on on-chain data and standard balance queries. A case study on 400 protocols shows that our estimations align with published figures for 46.5% of protocols. Informed by these findings, we discuss design guidelines that could facilitate a more verifiable, standardized, and explainable TVL computation.

Index Terms—Decentralized Finance, DeFi, Total Value Locked, TVL, Ethereum.

I. INTRODUCTION

The core value proposition of Decentralized Finance (DeFi) lies in its transparency and reliance on permissionless on-chain infrastructure [1]. As Total Value Locked (TVL) has become a primary metric for assessing the economic scale of DeFi — measuring the value of assets deposited in smart contracts — it is essential that its calculation upholds the same principles, remaining fully anchored in on-chain data and fully reproducible calculations. In spite of this, TVL computation today suffers from a lack of standardization and is difficult to verify independently.

In practice, TVL estimates are published by aggregators like DappRadar, Stelareum, and DeFiLlama, which typically adopt a community-driven approach. The latter, for instance, allows anyone to integrate a new DeFi protocol and its TVL computation methodology by developing a protocol-specific plugin and publishing it in an open-source GitHub reposi-

tory [2]. Contributors are encouraged to use only on-chain data for TVL calculations [3]. However, some plugins rely on data from external services and use self-defined functions for on-chain computations. Moreover, there is variability in how the values of assets deposited in contract accounts are calculated. At the end of 2024, Ethereum TVL estimates published by different aggregators vary from approximately \$80 bln to \$190 bln, indicating that remarkable differences exist in the methodologies used for TVL computation.

The need for independent verifiability has been demonstrated in cases where DeFi developers on the Solana blockchain deliberately designed their protocols to inflate the actual value of deposited assets, ultimately manipulating TVL figures [4]. The necessity for a standardized approach has become evident with the recognition that crypto deposits can be double-counted across DeFi protocols [5]–[7]. This issue has recently been addressed by proposing an alternative metric, Total Value Redeemable (TVR), which refines TVL by excluding cryptoassets that derive their value from underlying cryptoassets [8]. However, a comprehensive understanding of the methodologies used to compute TVL is still lacking.

In this paper, we aim to fill this gap by conducting a comprehensive measurement study to examine how TVL is computed in practice and to assess the extent to which its value can be recomputed and verified using on-chain data only. Our contributions and key empirical findings are as follows:

- 1) We develop and apply a measurement instrument to 939 DeFi protocols on the Ethereum chain. Our findings reveal that (i) 10.5% of them rely partially or entirely on data from external services to compute TVL, impeding full reproducibility; (ii) while the majority of protocols (78.6%) use standard balance queries, a subset of non-standard, self-defined balance functions ($N = 68$) is employed; (iii) the usage of these alternative queries has declined from 28.2% in January 2023 to 8.9% in January 2024; (iv) 240 balance queries executed on the same contracts and tokens are repeated on multiple protocols.
- 2) We introduce *verifiable Total Value Locked* (vTVL), a

metric assessing to what extent individual projects' TVL can be reconstructed using blockchain data and standard balance queries, and the *Discrepancy Ratio*, to quantify differences between published data and our estimates. A case study on 400 protocols shows that discrepancies are negligible for 23.5%, and estimations align with published figures for another 23%.

- 3) We propose design guidelines, informed by the challenges we identified, that could lead to a more verifiable and standardized TVL computation: (i) compute TVL from on-chain sources; (ii) publish protocol-specific contracts and tokens lists; (iii) favor standard balance methods whenever possible; (iv) publish the token categorizations used; and (v) define common standards for protocol selection criteria and version management.

As DeFi continues to mature, it is essential to rely on clear and reproducible metrics, especially when these are heavily used for business and investment decisions. Standardization is crucial in this regard, and verifiability should be part of a broader discussion on auditing within the DeFi ecosystem.

Section II introduces background and related work; Section III discusses how TVL is currently computed, while Section IV reports the case study. Section V and VI respectively discuss design guidelines and conclusions. Our data and code are available at https://github.com/PietroSaggese/TVL_Study.

II. BACKGROUND AND RELATED WORK

A. Cryptoassets: derivative and non-derivative tokens

Cryptoassets represent and facilitate transfer of value in a Distributed Ledger Technology (DLT) [1]. They can be categorized according to different factors [9]–[11]. In our context, we distinguish between derivative and non-derivative tokens [8]. Non-derivative or plain tokens are those without an underlying cryptoasset. They include native tokens such as Bitcoin, governance tokens, and non-crypto-backed (NCB) stablecoins, i.e., cryptoassets like USDC whose value is pegged to a target currency and whose reserves are not composed of cryptoassets. Derivative tokens represent instead a receipt token that grants a claim on an underlying cryptoasset. They (non-exhaustively) include liquidity pool (LP) tokens [12], interest-bearing tokens [13], [14], liquid staking tokens (LSTs) [15], and other financial products like synthetic tokens and tokenized baskets of assets. Derivative tokens also include crypto-backed stablecoins such as DAI.

B. Total Value Locked

DeFi protocols operate on a peer-to-pool model: investors deposit their cryptoassets into accounts that pool the invested funds to offer financial services [12], [16]. Total Value Locked is the primary metric for assessing the performance of DeFi protocols and the broader DeFi ecosystem, and it is computed as the aggregate value of cryptoassets deposited in the smart contracts that constitute one protocol.

More formally, let $\mathcal{P} = \{p_1, \dots, p_n\}$ be the set of all DeFi **projects** or **protocols** and $\mathcal{A} = \{a_1, \dots, a_k\}$ be the set of all **cryptoassets** deployed on a DLT (we remove time subscripts

for ease of notation). For one project p , $\mathcal{C}^p = \{c_1^p, \dots, c_m^p\}$ is the set of project-specific contracts. The association of a contract to a project must be *mutually exclusive*, i.e. $\mathcal{C}^p \cap \mathcal{C}^{p'} = \emptyset$. For each contract $c \in \mathcal{C}^p$, a vector $\mathbf{q}$ of length k indicates the amount of tokens locked into the contract, and the price of each token $a \in \mathcal{A}$ is π_a , denominated in a common currency. We then construct the matrices

$$Q^p = \begin{bmatrix} q_{c_1^p, a_1}^p & \cdots & q_{c_1^p, a_k}^p \\ q_{c_2^p, a_1}^p & \cdots & q_{c_2^p, a_k}^p \\ \vdots & \ddots & \vdots \\ q_{c_m^p, a_1}^p & \cdots & q_{c_m^p, a_k}^p \end{bmatrix}, \quad \Pi = \begin{bmatrix} \pi_1 \\ \pi_2 \\ \vdots \\ \pi_k \end{bmatrix}, \quad V^p = \begin{bmatrix} v_1^p \\ v_2^p \\ \vdots \\ v_m^p \end{bmatrix}$$

Where $V^p = Q^p \cdot \Pi$. Then TVL^p , denoting the value locked in a protocol, and TVL , denoting the value deposited in a DLT, are respectively computed as:

$$TVL^p = \sum_{i=1}^m v_i^p, \quad TVL = \sum_{p=1}^n TVL^p$$

C. TVL aggregators

Several aggregators publishing DeFi TVL estimations exist, e.g. DappRadar [17], Stelareum [18], DeFi Pulse [19], Coingecko [20], and DeFiLlama [21]. The estimations they report ultimately rely on on-chain token volumes and price information, but the methods used to compute these figures and the completeness of the related documentation vary greatly.

DeFiLlama is one of the most comprehensive aggregators in terms of TVL information disclosure [8]. Its core infrastructure, the DefiLlama-Adapters GitHub repository [2], enables the community to integrate new protocols through plugins, supposedly developed by the protocols' own maintainers, and is public — so anyone can observe how TVL is computed. However, since the plugins are contributed directly by the community, transparency and reliability concerns may arise. The DeFiLlama maintainers discourage the use of data not directly sourced from cryptocurrency nodes (with the exception of exchange rates from Coingecko) [3], but the framework does not impose strict limitations on off-chain data sources. Even when utilizing on-chain data only, the plugin creators can use self-defined, not documented on-chain functions to compute TVL, and the projects do not publish explicitly the contracts and tokens utilized to compute TVL. This information is implicit in the plugin code and no further documentation is provided by protocols. Moreover, the computing code may depend on specific implementations and therefore vary across time. There is also no description informing whether plugin contributors are actual protocol maintainers or other users.

Other aggregators provide more limited documentation, and in some cases, we could not find their TVL-computing code. Some aggregators require DeFi projects to submit a request to be included in TVL calculations, publishing only the final TVL values [18], [19]. These differences can introduce self-selection bias, i.e. the reported TVL may differ consistently across platforms if protocols share information selectively.

Finally, some platforms publish information related to their asset selection criteria, describing what tokens are included

in TVL computations [19], [21]. To date, this process is not standardized. Notably, also DeFiLlama’s framework allows to select or exclude, e.g., governance tokens, borrowed tokens, and others [6]; its TVL estimations at the end of 2024 for Ethereum range from 80 to 190 bln\$.

D. Related work

Surprisingly, despite the limitations discussed above and the central role TVL plays in DeFi, little research has systematically examined how TVL is computed. This is especially important considering that several studies base their analyses on TVL, e.g., to investigate DeFi growth rates [22] or its relation to ETH returns [23], used it as a variable in econometric [24]–[26] and machine learning models [27], or examined it in the context of relevant DeFi indicators [28], [29].

One major limitation of TVL identified by the academic community is the double counting problem [5], [30]. A first solution to this issue was proposed by Luo et al. [8], who focused on asset selection criteria and devised a novel metric to address double counting. *Total Value Redeemable* (TVR) is defined as the value that can be ultimately redeemed from a DeFi ecosystem, i.e., the sum of non-derivative tokens deposited in DeFi protocols.

Instead, we are not aware of any prior work assessing comprehensively the reproducibility and verifiability of TVL estimates. Our study fills this gap and complements existing literature [8] by investigating empirically how TVL is effectively computed, what methodologies are used, and the extent to which TVL is reproducible using available on-chain data.

III. TVL COMPUTATION IN PRACTICE

In this section, we examine how TVL is computed in practice, what data sources are used and what potential challenges to reproducibility and standardization arise from those design decisions. We focus on the projects listed on DeFiLlama for the reasons discussed in Subsection II-C (in principle, the analysis could be conducted on other aggregators publishing their TVL-computing code). We restricted the blockchain data collection to Ethereum as it plays a major role in DeFi with 66% of total TVL according to DeFiLlama. We expect that our findings can be generalized to most alternative chains that are EVM compatible and mirror Ethereum’s ecosystem. The pipeline of the analysis is shown in Figure 1.

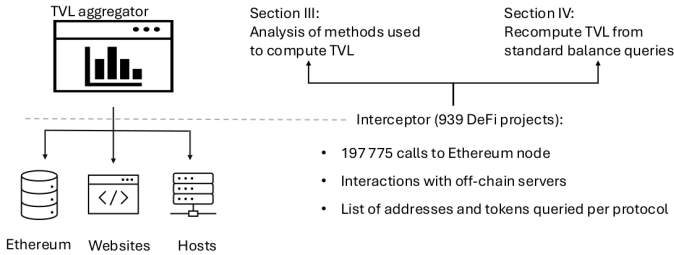


Fig. 1: **Analysis pipeline.** For each protocol, we execute the plugin code provided by the TVL aggregator and record all interactions with both on-chain and off-chain sources.

Method	Count	Num. of protocols	Returns the ...
balanceOf	102 655	641	...account balance of another account (owner address) [32].
eth_getBalance	1696	170	...ETH balance of the account of a given address [33].
totalSupply	2577	154	...total token supply [32].
getReserves	3016	121	...reserves of token0 and token1 used to price trades and distribute liquidity [34].
token0	1040	94	...address of the first pair token [34].
token1	1040	94	...address of the second pair token [34].
symbol	1835	64	...symbol of the token. [32].
allPairsLength	46	45	...total number of pairs created through the factory [35].
underlying	1613	43	...address of the underlying token [36].
totalAssets	236	26	...total quantity of all assets under control of a Vault [37].

TABLE I: **Most frequently queried functions.** The *balanceOf* method is the most commonly used in TVL computations, along with *eth_getBalance*. Other functions, such as *totalAssets*, offer alternative approaches to retrieving balances.

A. Measurement method and summary statistics

To conduct our analyses, we developed a data extraction pipeline that proxies and systematically records all interactions between the DeFiLlama tooling and its runtime environment during TVL computation. This encompasses all calls directed towards the Ethereum node software, as well as interactions with external hosts and web services. Our instrumentation records on-chain interactions for arbitrary combinations of Ethereum block height, commit, and chains (more details on this procedure are reported in Appendix A).

Following the recording process, we filter and interpret the collected on-chain data to identify the functionalities used and accounts accessed per protocol. To interpret the data, we map the signatures of the called functions either directly when stored or using the Ethereum Signature Database 4byte [31].

Our main dataset consists of the interactions between the DeFiLlama repository and its environment, collected on January 4 2024 (commit 6764756f9270ab6a3047c06c13c0b1b2d32a3247). It includes 939 projects deployed on Ethereum out of 3494 total projects. It comprises their interactions with external servers and the blockchain infrastructure, the latter resulting in 197 775 proxied calls. To validate consistency, we repeated the collection on four other dates (04 Jan 2023, 04 Apr 2023, 04 Jul 2023, 04 Oct 2023).

Table I reports the most relevant functions called on-chain, ranked by number of protocols using them and complemented by a short description of their use. As expected, the most frequently called methods are balance queries for ERC-20 compatible tokens (*balanceOf*) and Ether (*eth_getBalance*), but alternative functions exist (e.g., *totalAssets*). *BalanceOf* is primarily queried on wETH, stablecoins, governance tokens, and staked tokens (see Appendix A for further details). Other methods are used to query supplementary information, e.g., the token address or its symbol (*token0*, *token1*, *symbol*), are

used to price trades and distribute liquidity (*getReserves*), or retrieve token information (*underlying*, *totalSupply*).

B. Reproducibility and reliance on off-chain data

Relying on public and transparent on-chain information entails a number of advantages compared to off-chain data sources. The latter is easier to tamper with, creates more dependencies not in control of the user, and is less transparent, since one has limited visibility into web services’ internal operations. We thus run each protocol’s computation code and investigate whether the infrastructure relies solely on on-chain data or also exploits external hosts. We also document whether the computation executes correctly or if errors occur.

Figure 2 represents graphically the space of 939 DeFiLlama projects analyzed. For the largest group ($N = 729$), the procedure is executed without errors and no external hosts are used. While 73 projects utilized both external hosts and on-chain data sources, for 26 protocols we observe interactions with external hosts but no direct on-chain data could be retrieved; since no errors occurred in their collection procedure, we infer that they solely rely on external services. The TVL of the remaining protocols is either computed despite errors being produced during computations ($N = 28$) or not computed at all ($N = 62$). Finally, 22 projects did not produce any interaction and were discarded.

Among the 64 external servers identified, some pose a smaller threat to verifiability, e.g., TheGraph, an indexing protocol for accessing blockchain data; others appear to be associated with third parties or specific protocols. The errors are mostly related to technical failures such as the block height provided not being accepted, missing fields or parameters, and asynchronous operations that were not completed successfully. Further details are reported in Appendix B.

Arguably, TVL is computable with on-chain data alone, but in practice we find several instances where other data sources are used. Despite the clarity of the DeFiLlama maintainers’ objectives, certain protocols (10.5%, including Uniswap V1 and V2) rely partially or entirely on data extracted from external servers for computations, partially compromising verifiability.

C. Heterogeneity of on-chain interactions

Even when utilizing on-chain data only, challenges to explainability may arise. Projects use a wide set of functions alternative to standard *eth_getBalance* and *balanceOf* queries to acquire on-chain balance data. These could hinder interpretation and introduce sources of tampering or inaccuracies if their logic is unclear. We thus focus on the 197 775 calls directed to an Ethereum node and analyze to what extent the methods used to compute TVL are heterogeneous and the computation approach standardized across protocols.

First, to study if certain functions play an outsized role or if anomalous patterns emerge, we fit to a power-law distribution the frequency of occurrence both of the functions called and of the token calls in *balanceOf* functions. Analyzing the frequency of occurrence of the functions called, we observe that *balanceOf* emerges as an extreme value with respect to the

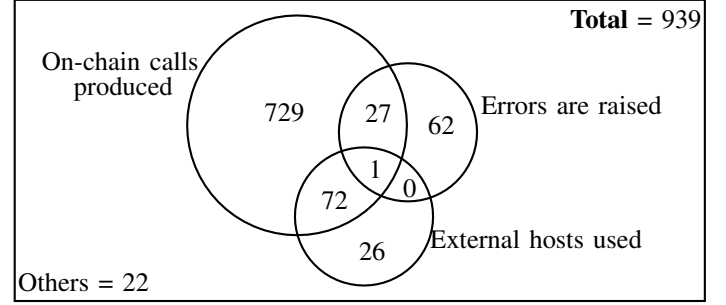


Fig. 2: **Projects using off-chain in addition to on-chain sources.** Projects are categorized based on their reliance on external hosts and whether any errors occurred during data collection. While for 729 projects, on-chain calls were executed without errors and without the use of external sources, 99 projects depend on external servers.

power-law fit, further highlighting its central position in TVL measurement. In general, the results of the fit are consistent with the behavior of a heavy-tailed distribution; the estimated α range from 1.63 to 1.85, coherently with findings of previous network studies on Ethereum blockchain data [38], [39].

Next, we devise an approach to identify the functions alternative to *balanceOf* and *eth_getBalance* (used by 78.6% of protocols) whose name and description indicate that they are likely alternative functions self-defined by projects to compute TVL. We conduct a keyword-based search through regular expressions on the function signature names. We find 68 alternative functions, used by 14.2% of protocols, that plausibly contribute to TVL computation. Out of these, four functions, called 94 times only, have name *balanceOf*, but different signatures than the standard ERC-20. The remaining protocols either rely on external hosts or exploit functions ($N = 88$) that are not clearly matched to a balance-querying functionality after a manual check. While standard functions are simpler to interpret, the alternative functions are specific to a few or just one project, and their logic is hard to interpret without investigating the code in depth. One illustrative example is represented by Lido, whose TVL is computed through one single function (*getTotalPooledEther*) that likely returns as output the total protocol TVL.

In summary, while most protocols (78.6%) use standard token balance queries, a subset of alternative functions ($N = 68$) is employed; however, this makes it harder to understand how TVL is computed for protocols utilizing such functions. Further details on the network analysis and on the detection of the alternative functions are reported in Appendix C.

D. Changes in TVL computation methods

Another thread to verifiability and consistency are changes to the code itself. TVL computation relies on self-reports from DeFi protocols maintainers, and evidence of manipulations to purportedly inflate TVL exists [4], [7]. More generally, computations may depend on specific implementations and therefore vary across time. As our pipeline enables the data

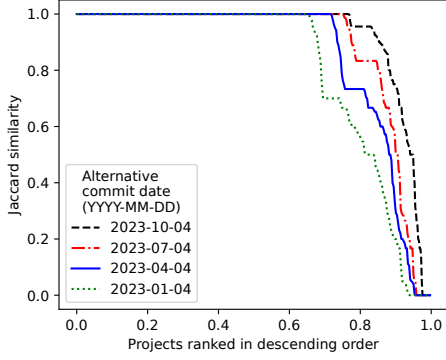


Fig. 3: **Pairwise Jaccard similarity, computed for each protocol on the set of *balanceOf* calls queried in the main and four older commits.** Each line reports the values relative to one specific commit. Projects are ranked in descending order based on their similarity score (ranging from 0 to 1). A value of 1 indicates no changes among the compared sets.

collection for various commits, we repeat the data gathering at four alternative dates (4th of January, April, July, and October 2023) and analyze changes in the Ethereum queries per protocol. We focus on *balanceOf* for the interpretability of the associated calls and measure to what extent projects called different addresses and tokens over time.

To do so, for each protocol we compare the set of *balanceOf* calls executed in the main commit (Jan 4 2024) to the set of *balanceOf* calls executed in each older commit analyzed¹. We quantify the differences between sets as the pairwise Jaccard similarity; values range from 0 to 1, and a value of 1 indicates no changes among the compared sets. Figure 3 shows the results: each line reports the values relative to one specific commit. Projects are ranked in descending order, based on their Jaccard similarity score. We observe that in each older commit the set of calls does not change for most projects. The average Jaccard index ranges from 0.93% (Oct 2023, black dashed line) to 0.89%, 0.86% and 0.81%, respectively in Jul, Apr, Jan 2023. The Jaccard similarity decreases as expected with time, but moderately. Protocols typically do not modify the TVL computation code significantly across time. This finding has a two-fold interpretation: on the one hand, computation is relatively stable over time and thus less challenges arise from this perspective; on the other hand, it is possible that protocols are not systematically updating their plugins despite smart contract changes being implemented.

Similarly, we measure how the use of functions alternative to *balanceOf* varies across commits. We find that the ratio between the number of alternative function queries over that

¹We only compare protocols that existed at both commit dates and exclude those that used hosts or raised errors during the data collection. We note that we do not investigate commits executed earlier than 2023 because the number of projects that can be compared decreases with time (from nearly 500 between Oct 2023 and Jan 2024 to less than 300 between Jan 2023 and Jan 2024) and because our pipeline is optimized to capture data on recent implementations of the repository.

of *balanceOf* and *eth_getBalance* calls is higher in older commits (28.2% in Jan 2023, 22.7% in Apr 2023, 19.9% in Jul 2023, and 12.2% in Oct 2023, against 8.9% in Jan 2024). We interpret this as a sign that heterogeneity in computation methods has reduced over time. Appendix D reports additional data, such as the evolution in time of the number of standard and non-standard calls, the full identifiers of older commits, and additional results, including the Jaccard similarity scores on all protocols and in absolute numbers.

E. Equivalent balance queries linked to multiple protocols

Double counting represents a major issue in the computation of financial metrics like TVL. It occurs when the value of an asset is counted more than once, leading to an inflated representation of total assets within the system. Previous research [8] introduced the Total Value Redeemable as an alternative to TVL and partly addressed the problem by counting the value of non-derivative tokens only. From a technical perspective, double counting can also occur if a smart contract included in the TVL calculation is not exclusively associated with one project, causing the corresponding TVL to be counted multiple times.

We thus search for balance queries that are not exclusively linked to a single protocol, thus potentially leading to such double counting. Notably, we identify 230 *balanceOf* and 10 *eth_getBalance* calls that appear to be associated with different protocols on the same input smart contract and for the same token address. A possible explanation for this finding is that these contracts are managing interactions across protocols². We measure and discuss the magnitude of this phenomenon in economic terms in the next section. Appendix E reports a full list of the addresses and tokens involved.

IV. CASE STUDY: TVL RECONSTRUCTION FROM ON-CHAIN DATA AND vTVL

In principle, TVL can be calculated entirely on public, immutable blockchain information: this may provide a solution to some of the challenges emerged in TVL computation affecting its reproducibility and verifiability. Building on this, we conduct a case study to assess to what extent the TVL of individual projects can be recomputed and verified solely relying on on-chain data and standard balance queries. We call the resulting metric the verifiable Total Value Locked (vTVL) of a DeFi project. This serves as a starting point for discussing a set of recommendations and standardization attempts to improve TVL computation.

A. TVL reconstruction approach

Having access to the functionalities and accounts queried on-chain through the DeFiLlama infrastructure, we can acquire a set of addresses per protocol containing deposited assets and provide further insights on the TVL associated with each protocol, solely relying on blockchain data (assuming that addresses called during computation contribute to the project

²While we assume that balances queried during computation directly contribute to the projects TVL, we acknowledge the possibility that they don't.

TVL). To have a homogeneous and standardized representation, we focus on *eth_getBalance* and *balanceOf* queries. We obtain a list of addresses contributing towards the locked value for each project, along with a compilation of tokens ($N = 12246$) in which all projects hold value. For each protocol, we extract cryptoassets quantities and prices on-chain. We query the state of protocol-specific addresses for their associated tokens and extract historical monthly balance information from Jan 1st 2021 to Feb 1st 2024. We price tokens by extracting exchange rate information from Uniswap V2 DEX liquidity pools [40]; in total, using this approach, we could price 942 tokens.

Having granular information on tokens deposited into contracts, we can investigate TVL composition and increase control over selected assets. We categorize tokens following the distinction into non-derivative and derivative tokens [8] and distinguish seven categories: Ether and its wrapped token wETH, wrapped BTC (wBTC), non-crypto-backed stablecoins, crypto-backed stablecoins, governance tokens, derivative tokens, and others. We note that we report separately the balance queries that, as discussed in Subsection III-E, are non-exclusively associated with one protocol and therefore potentially contribute to double counting, as we could not disentangle which project they should be associated with.

We thus recompute the value held by each protocol and call these estimates the verifiable Total Value Locked (vTVL). For comparison, we also download historical TVL values³ published per protocol from the DeFiLlama API service [41]. We compute for each project the *Discrepancy Ratio*, defined as the average ratio of the vTVL estimations over the data posted through the DeFiLlama APIs, normalized by subtracting one. A value of zero indicates perfect correspondence, while -1 indicates that vTVL equals zero. To interpret this metric, we recall that our estimations are an underestimation rather than an overestimation of the reported figures. We stress that the discrepancies should not be interpreted as a measure that protocols are inflating values; rather, they indicate to what extent we are able to independently verify the reported figures. Further details on this are given in Appendix F.

B. Case study: analysis and results

We analyze 400 protocols with at least one *eth_getBalance* or *balanceOf* recorded call and for which we could gather off-chain API data. Panels (a) to (e) of Figure 4 show the results for five protocols selected for their relevance in the DeFi ecosystem: in order, Aave (v2 and v3), Compound-v3, dYdX, Maker, and Uniswap-v3. Each panel shows a stacked plot of the vTVL divided by token categories as discussed above. The black line represents the (total) TVL value published on DeFiLlama. We observe that vTVL is mostly composed of ETH/wETH, wBTC, and non-crypto-backed stablecoins. In most cases, the data reconstructed from on-chain and off-chain

data are consistent; for some projects, the data match almost perfectly, while for others, we observe a partial discrepancy, but their overall trend is consistent. The discrepancies can arise for varying reasons, e.g., the use of alternative methods to compute balances, the reliance on external hosts to produce values (e.g. Maker) or the presence of errors during the interception procedure (e.g. Uniswap V3), but also for the lack of price data for certain tokens. Further details are reported in Appendix F and limitations are discussed in Section V.

Panel (f) reports instead the evolution over time of the value held in the contract accounts identified in Subsection III-E for the balance queries that are repeated over multiple protocols, thus potentially contributing to double counting. Notably, the value reached a peak of almost 16bln\$ at the end of 2021: double counting is a threat to a correct interpretation of the TVL metric also at the infrastructure level, and its potential impact is economically relevant.

Finally, to obtain a broader understanding of the entire ecosystem, we compute for each project the *Discrepancy Ratio*. Figure 5 shows the results. Each dot represents a project and its size is proportional to the amount of TVL they hold (according to off-chain estimations). Projects are ranked in descending order with respect to the Discrepancy ratio. The ones shown in Figure 4 are labeled. For 94 projects (23.5%), the difference lies within ± 0.05 , indicating almost perfect consistency, while for 186 projects (46.5%), the difference lies within ± 0.5 , indicating that figures are aligned but discrepancies exist. In 98 cases the ratio is either larger than 1 (top left) or equal to -1 (bottom right), indicating large discrepancies. The latter are mostly small and less relevant protocols.

In summary, the case study shows that with vTVL, which reproduces TVL using solely on-chain data and interpretable balance queries, we can independently verify just a part of the reported TVL. Discrepancies between our estimations and published data are close to zero for about one quarter of protocols (23.5%) and are aligned for about half (46.5%). Concurrently, the study shows that it is possible to follow a more transparent approach for computing TVL. The approach we used for vTVL removes potential off-chain sources of tampering, minimizes heterogeneity in computation methods, reveals patterns that can only be investigated on-chain, like the association of one contract to multiple protocols, and enables control over the selected assets, ultimately improving reproducibility, verifiability, transparency and interpretability.

V. DISCUSSION: TOWARDS TVL STANDARDIZATION

TVL today remains largely non-standardized and its computation open source to third parties. Against this backdrop, informed by the challenges identified in TVL computation and the case study conducted, we discuss a series of recommendations that may help guide the design of reproducible, verifiable, transparent and interpretable TVL estimations.

First, **TVL can and should be computed from available on-chain sources**. As discussed in Sections III and IV, relying on external services may hinder verifiability and reproducibility, raise concerns about transparency and create opportunities

³The API data are reported by DeFiLlama distinguished by chain and type. We include all values associated with the Ethereum blockchain (columns ‘Ethereum’, ‘Ethereum-borrowed’, ‘Ethereum-pool2’, ‘Ethereum-staking’, ‘Ethereum-vesting’).

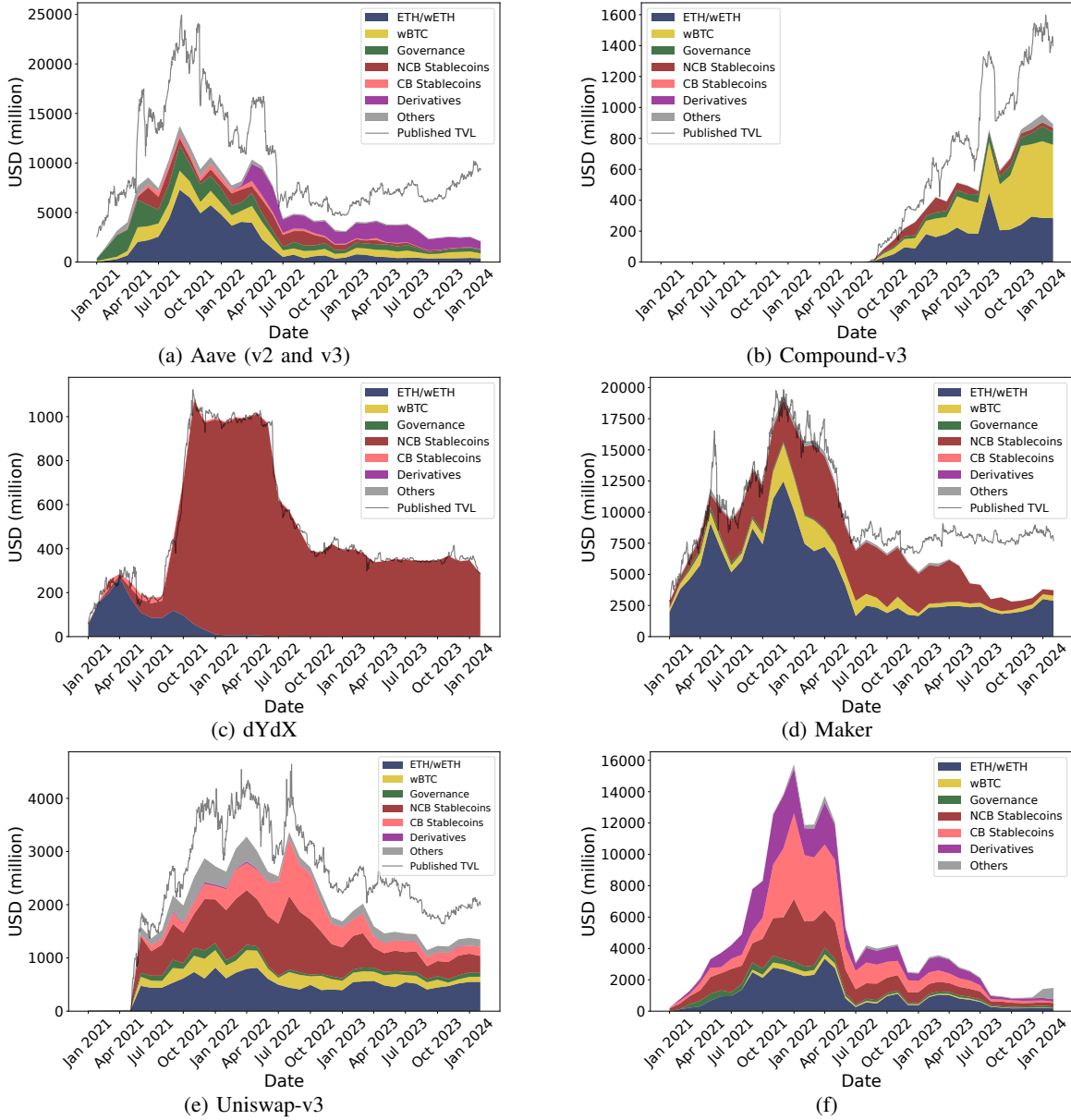


Fig. 4: **Verifiable Total Value Locked (vTVL)**. We query on-chain information for relevant DeFi protocols — Aave v2 and v3 (a), Compound v3 (b), dYdX (c), Maker (d), Uniswap v3 (e) — and compare it to published off-chain TVL data. Each plot represents the evolution in time of their vTVL as a stacked plot, divided in seven categories: Ether and wETH, wBTC, governance tokens, non-crypto-backed stablecoins, crypto-backed stablecoins, and uncategorized tokens. Panel (f) shows instead the evolution in time of the value of 240 *balanceOf* and *get_ethBalance* queries called on the same contracts and tokens but associated with different protocols (see Subsection III-E), potentially contributing to double counting.

for manipulation or double counting. Furthermore, third parties need to **know explicitly what protocol-specific smart contracts and associated tokens** are utilized to count value. This also allows to verify if the association of a contract to a project is unique or a potential source of double counting.

The use of **standard balance methods should be preferred over custom functions whenever possible**, in order to standardize computation methods and enhance interpretability of the metric. As discussed in Section III-A, the most common to-

kens are ERC-20 compliant and therefore it is straightforward to understand how their balance is quantified. If tokens are non-standard or rely on alternative functions, proper documentation is required to interpret results. A promising finding in this sense is the increased predominance over time of standard functions, which revealed a trend towards homogenization.

We propose that aggregators **publish their token categorizations** and enable third parties to independently **select which assets to include or exclude in computations**, with

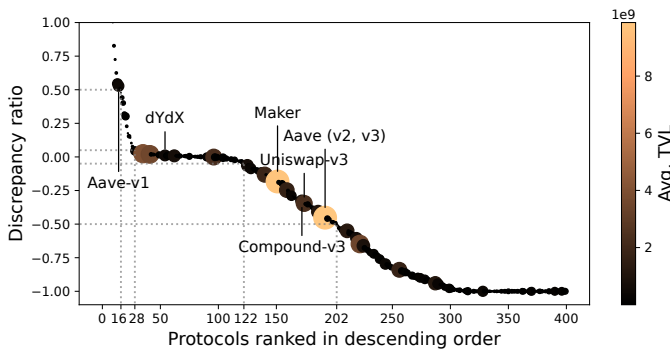


Fig. 5: **Differences between vTVL and published TVL.** Projects are dots ranked in descending order based on the *Discrepancy ratio*, i.e., the ratio of on-chain estimations to off-chain data, normalized by subtracting one.

particular attention to derivative and non-derivative tokens. While aggregators today allow to include or exclude certain tokens from computations, such as governance tokens, borrowed tokens in lending protocols, and others [6], we could not find a detailed list of what assets are included in each category. Furthermore, as indicated in previous research [8], derivative tokens are responsible for double counting, and this aspect needs to be taken into account. In our case study, we grouped tokens into categories and identified derivative tokens to provide deeper insights on TVL composition, allowing direct control on the assets selected for computation.

More broadly, it is also necessary to **define common standards for protocol selection criteria** and maintain a public list with included protocols. According to DeFiLlama APIs, the protocols we analyzed are grouped into 39 categories: the most common ones are DEXs, Yield, Lending, and Services (respectively $N = 78, 68, 55, 34$). Notably, $N = 28$ are labeled as CEXs. It is also arguable whether other categories belong to DeFi (Gaming, Oracle, Wallets) or what their purpose is (SoFi, Launchpad, Chain). Furthermore, relying on self-reporting also implies self-selection bias to a certain extent: some protocols might not be interested in reporting data to all TVL aggregators and viceversa. Related to protocol selection, it is important to **complement protocol-specific information with metadata for version management**. From the analysis in Section IV-A, it emerged that protocols plugins do not manage versions consistently. As protocols often deploy new versions, it is important to have a clear vision of what funds are deposited in each protocol version.

We acknowledge that future research might address some limitations of our study. First, our work is limited to the DeFiLlama infrastructure and to Ethereum alone, but DeFi is spread across DLTs; whilst we believe our findings can be generalized to other blockchains that replicate Ethereum, future analyses could pay specific attention to bridges and multi-chain protocols, and include other blockchains like Solana. Second, alternative computation methods to standard ones should be investigated in detail, to understand what

functionalities they offer and why they are used. Third, the token categorization needs improvement. Currently, there is no comprehensive method for identifying derivative tokens, nor a clear and shared definition. Similarly, the collection of token prices can be improved. We could not price all tokens in our dataset with our approach (but we did cover all the most relevant ones), and on-chain prices should be extracted from multiple DEXs and liquidity pools to account for low-liquidity scenarios. The price of certain tokens, like NFTs, might need to be complemented with exchange market price data. Fourth, relying only on on-chain sources is likely more costly and less efficient computationally. This approach might face resistance due to the varied nature of the platforms involved. Following the suggested approach, protocols might have to disclose more information than they do today. As this comes at a gain in terms of transparency and verifiability, it is important to further investigate how to balance these aspects.

Beyond our analysis, we note that TVR [8] is devised for application on an entire ecosystem and therefore helps standardize TVL at the ecosystem level. However, it remains still unclear how to conduct asset selection at the protocol level. Appendix G provides further analyses on TVL composition changes in relation to TVR and across protocol category, size, and time. Further research should focus on this aspect.

Finally, we remark that the verifiability and transparency of TVL computations is part of a broader discussion on DeFi-related risks and issues. Indeed, DeFi automation and the removal of human involvement has introduced or heightened certain risks, e.g. diminishing oversight and control, or paving the way for new types of intermediaries [42]. Just as TVL should be verifiable, proof-of-reserve systems are essential for stablecoins and cryptoasset trading platforms [43], [44]. Moreover, the need for governance makes some degree of centralization unavoidable [45], and structural aspects of the system contribute to the concentration of power in the hands of few developers [46], [47].

VI. CONCLUSIONS

In this work, we conduct a systematic study on 939 DeFi projects deployed in Ethereum. We first provide a comprehensive understanding of the methodologies currently used for TVL computation; next, we examine the extent to which TVL is reproducible and verifiable using available on-chain data by introducing a new metric, the verifiable Total Value Locked (vTVL). Informed by these analyses, we propose design guidelines and possible standardization attempts in the field.

TVL is a fundamental financial metric in the DeFi ecosystem. Reaching common standards and publishing verifiable figures is critical to obtaining a clear overview of the true dimensions of the DeFi ecosystem, informing correctly users' investment decisions, and guaranteeing fair competition across protocols. This also supports the need for greater financial transparency and accountability to address the expectations of diverse stakeholder groups, including users, investors, and regulators. This work provides several insights in this direction.

ACKNOWLEDGMENTS

The Complexity Science Hub researchers were partially funded by the Austrian security research program KIRAS of the Federal Ministry of Finance (BMF) under the project DeFiTrace (grant agreement number 905300) and the FFG BRIDGE project AMALFI (grant agreement number 898883).

REFERENCES

- [1] R. Auer, B. Haslhofer, S. Kitzler, P. Saggese, and F. Victor, "The technology of decentralized finance (defi)," *Digital Finance*, vol. 6, no. 1, pp. 55–95, 2024.
- [2] DeFiLlama, "GitHub repository - DeFiLlama-Adapters," available at: <https://github.com/DefiLlama/DefiLlama-Adapters>.
- [3] —, "Why we don't accept APIs," available at: <https://github.com/DefiLlama/DefiLlama-Adapters/discussions/432>.
- [4] D. Nelson and T. Wang, "Master of anons: How a crypto developer faked a defi ecosystem," <https://www.coindesk.com/layer2/2022/08/04/master-of-anons-how-a-crypto-developer-faked-a-defi-ecosystem/>.
- [5] J. Chiu, T. Koepl, H. Yu, and S. Zhang, "Understanding defi through the lens of a production-network model," Bank of Canada, Tech. Rep., 2023.
- [6] Coindesk, "Data provider defillama de-emphasizes double-counted crypto deposits after saber revelation," 2022, available at: <https://www.coindesk.com/business/2022/08/05/data-provider-defillama-de-emphasizes-double-counted-crypto-deposits-after-saber-revelation/>.
- [7] L. Nuzzi, A. L. Calvez, and K. Waters, "Understanding total value locked (tvL)," 2021, available at: <https://coinmetrics.substack.com/p/coin-metrics-state-of-the-network-0c0>.
- [8] Y. Luo, Y. Feng, J. Xu, and P. Tasca, "Piercing the veil of tvL: Defi reappraised," in *International Conference on Financial Cryptography and Data Security*. Springer, 2025.
- [9] M. Fröwis, A. Fuchs, and R. Böhme, "Detecting token systems on ethereum," in *International Conference on Financial Cryptography and Data Security*. Springer, 2019, pp. 93–112.
- [10] M. Di Angelo and G. Salzer, "Identification of token contracts on ethereum: standard compliance and beyond," *International Journal of Data Science and Analytics*, vol. 16, no. 3, pp. 333–352, 2023.
- [11] A. Moin, K. Sekniqi, and E. G. Sirer, "Sok: A classification framework for stablecoin designs," in *International Conference on Financial Cryptography and Data Security*. Springer, 2020, pp. 174–197.
- [12] J. Xu, K. Paruch, S. Cousaert, and Y. Feng, "Sok: Decentralized exchanges (dex) with automated market maker (amm) protocols," *ACM Computing Surveys*, vol. 55, no. 11, pp. 1–50, 2023.
- [13] L. Gudgeon, S. Werner, D. Perez, and W. J. Knottenbelt, "Defi protocols for loanable funds: Interest rates, liquidity and market efficiency," in *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*, 2020, pp. 92–112.
- [14] S. Cousaert, J. Xu, and T. Matsui, "Sok: Yield aggregators in defi," in *2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2022, pp. 1–14.
- [15] X. Xiong, Z. Wang, X. Chen, W. Knottenbelt, and M. Huth, "Leverage staking with liquid staking derivatives (lsds): Opportunities and risks," *arXiv preprint arXiv:2401.08610*, 2023.
- [16] T. A. Xu and J. Xu, "A short survey on business models of decentralized finance (defi) protocols," in *International Conference on Financial Cryptography and Data Security*. Springer, 2022, pp. 197–206.
- [17] DappRadar, "DeFi Rankings," available at: <https://docs.dappradar.com/rankings/defi-rankings>.
- [18] Stelareum, "Total Value Locked (TVL) in DeFi protocols," available at: <https://www.stelareum.io/en/defi-tvl.html>.
- [19] DeFiPulse, "DeFiPulse Total Value Locked (TVL) Methodology," available at: <https://docs.defipulse.com/methodology/tvl>.
- [20] Coingecko, "What Total Value Locked (Tvl) and Why Users Monitor This Metric," see: <https://www.coingecko.com/learn/total-value-locked>.
- [21] DeFiLlama, "DeFiLlama website," available at: <https://defillama.com/>.
- [22] V. Stepanova and I. Eriniš, "Review of decentralized finance applications and their total value locked," *TEM Journal*, vol. 10, no. 1, p. 327, 2021.
- [23] K. Shilov and A. V. Zubarev, "Return factors of ether cryptocurrency: On chain metrics and defi," *Available at SSRN 4432586*.
- [24] Y. Maouchi, L. Charfeddine, and G. El Montasser, "Understanding digital bubbles amidst the covid-19 pandemic: Evidence from defi and nfts," *Finance Research Letters*, vol. 47, p. 102584, 2022.
- [25] L. Zhou, X. Xiong, J. Ernstberger, S. Chaliasos, Z. Wang, Y. Wang, K. Qin, R. Wattenhofer, D. Song, and A. Gervais, "Sok: Decentralized finance (defi) attacks," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 2444–2461.
- [26] F. Şoiman, G. Dumas, and S. Jimenez-Garcés, "The return of (i) defix," *arXiv preprint arXiv:2204.00251*, 2022.
- [27] S. Fan, T. Min, X. Wu, and C. Wei, "Towards understanding governance tokens in liquidity mining: a case study of decentralized exchanges," *World Wide Web*, pp. 1–20, 2022.
- [28] D. Metelski and J. Sobieraj, "Decentralized finance (defi) projects: A study of key performance indicators in terms of defi protocols' valuations," *International Journal of Financial Studies*, vol. 10, no. 4, p. 108, 2022.
- [29] T. Katona, "Decentralized finance: the possibilities of a blockchain "money lego" system," *Financial and Economic Review*, vol. 20, no. 1, pp. 74–102, 2021.
- [30] K. Saengchote *et al.*, "Where do defi stablecoins go? a closer look at what defi composability really means," *Available at SSRN 3893487*, 2021.
- [31] 4byte, "Ethereum Signature Database," see <https://www.4byte.directory>.
- [32] Ethereum.org, "ERC-20: Token Standard Improvement Proposal," see <https://eips.ethereum.org/EIPS/eip-20>.
- [33] —, "Ethereum JSON-RPC API," see https://ethereum.org/en/developers/docs/apis/json-rpc/#eth_getbalance.
- [34] Uniswap, "Functionalities documentation," see <https://docs.uniswap.org/contracts/v2/reference/smart-contracts/pair>.
- [35] —, "Factory contract documentation," see <https://docs.uniswap.org/contracts/v2/reference/smart-contracts/factory>.
- [36] Etherscan, "AaveV2Provider contract," see <https://etherscan.io/address/0x7ac6859e69d6549b39a8367097d7ae5fbff5951e#readContract>.
- [37] —, "Vyper contract," see <https://etherscan.io/address/0xba3cfea6514cf5acddef3167df0b7a4337751bc#code>.
- [38] S. Kitzler, F. Victor, P. Saggese, and B. Haslhofer, "Disentangling decentralized finance (defi) compositions," *ACM Transactions on the Web*, vol. 17, no. 2, pp. 1–26, 2023.
- [39] X. T. Lee, A. Khan, S. Sen Gupta, Y. H. Ong, and X. Liu, "Measurements, analyses, and insights on the entire ethereum blockchain network," in *Proceedings of The Web Conference*, 2020, pp. 155–166.
- [40] L. Heimbach, Y. Wang, and R. Wattenhofer, "Behavior of liquidity providers in decentralized exchanges," *arXiv preprint arXiv:2105.13822*, 2021.
- [41] DeFiLlama, "APIs," available at: <https://defillama.com/docs/api>.
- [42] N. Carter and L. Jeng, "Defi protocol risks: The paradox of defi," *Regtech, supotech and beyond: innovation and technology in financial services' riskbooks-forthcoming*, vol. 3, 2021.
- [43] B. Eichengreen, M. T. Nguyen, and G. Viswanath-Natraj, "Stablecoin devaluation risk," *WBS Finance Group Research Paper*, 2023, available at SSRN: <http://dx.doi.org/10.2139/ssrn.4460515>.
- [44] P. Saggese, E. Segalla, M. Sigmund, B. Raunig, F. Zangerl, and B. Haslhofer, "Assessing the solvency of virtual asset service providers: are current standards sufficient?" *Applied Economics*, pp. 1–16, 2024.
- [45] J. F. Doerr, A. Kosse, A. Khan, U. Lewrick, B. Mojon, B. Nolens, and T. Rice, "Defi risks and the decentralisation illusion," *BIS Quarterly Review*, vol. 21, 2021.
- [46] C. Fracassi, M. Khoja, and F. Schär, "Decentralized crypto governance? transparency and concentration in ethereum decision-making," *Transparency and Concentration in Ethereum Decision-Making*, 2024.
- [47] S. Kitzler, S. Ballelli, P. Saggese, B. Haslhofer, and M. Strohmaier, "The governance of decentralized autonomous organizations: A study of contributors' influence, networks, and shifts in voting power," in *International Conference on Financial Cryptography and Data Security*. Springer, 2024, pp. 313–330.
- [48] A. D. Broido and A. Clauset, "Scale-free networks are rare," *Nature communications*, vol. 10, no. 1, pp. 1–10, 2019.
- [49] A. Clauset, C. R. Shalizi, and M. E. Newman, "Power-law distributions in empirical data," *SIAM review*, vol. 51, no. 4, pp. 661–703, 2009.
- [50] Coingecko, "Top Crypto Categories By Market Cap," available at: <https://www.coingecko.com/en/categories>.
- [51] Coinmarketcap, "Cryptocurrency Sectors by 24h Price Change," available at: <https://coinmarketcap.com/cryptocurrency-category/>.

A. Implementation Details

To investigate how TVL is technically computed in practice, we devised a data extraction pipeline that captures interactions of the DeFiLlama Adapters GitHub repository with its environment (e.g., nodes, websites, etc.) during TVL computation. In this Section, we provide additional implementation details on how to intercept and record such interactions.

To integrate a project into DeFiLlama, users need to develop plugins that contain the logic for computing TVL. The DeFiLlama Adapters GitHub repository provides an SDK to simplify this integration process. Once the project is successfully integrated, a dedicated folder is created within the repository, containing the protocol plugin and the necessary information to compute TVL.

To intercept the interactions, we instrument the runtime environment and execute each project’s plugin at the specified Ethereum block height and DeFiLlama commit. Refer to <https://github.com/mswjs/interceptors> for details on how we intercept all HTTP calls made by the Node.js process. In this way, we systematically capture all interactions (via http calls) with the environment involved in generating the TVL for a specific project. This notably encompasses all calls directed towards the Ethereum node software, as well as interactions with external hosts.

Table II reports additional information on the calls directed to the Ethereum node. Specifically, it shows the most common tokens queried in *balanceOf* calls. TVL is computed mostly on wETH and wBTC (respectively 10 984 and 1415 queries), on stablecoins (USDC, 4236; USDT, 2197; DAI, 2144), governance tokens (UNI, 891; COMP, 790; AAVE, 777), and staked tokens (stETH; 654).

Address	Symbol	Count
0xC02aaA3...	WETH	10 984
0xA0b8699...	USDC	4236
0xdAC17F9...	USDT	2197
0x6B17547...	DAI	2144
0x2260FAC...	WBTC	1415
0x5149107...	LINK	1002
0x1f9840a...	UNI	891
0xc00e94C...	COMP	790
0x7Fc6650...	AAVE	777
0xD533a94...	CRV	760
0x9f8F72a...	MKR	715
0x6B35950...	SUSHI	690
0xae7ab96...	stETH	654
0x7D1AfA7...	MATIC	646
0x1111111...	1INCH	616
0x408e418...	REN	615
0x4Fabb14...	BUSD	570
0x0F5D2fB...	MANA	564
0xc944E90...	GRT	556
0x0000000...	TUSD	549

TABLE II: **Most frequently called tokens in *balanceOf* functions.** Wrapped ETH and BTC, stablecoins, governance and staked tokens play a primary role.

B. Reproducibility and reliance on off-chain data

In this appendix, we provide additional information on the servers and errors that were detected during TVL computation. Table III reports a list of the documented errors categorized by typology. The most occurring ones are related to the block height provided not being accepted, or are caused by a collection of asynchronous operations that did not complete successfully. Other errors can be reconducted to the lack of missing fields or parameters and other technical problems. Table IV reports a full list of the detected servers. While the one occurring most frequently is part of TheGraph infrastructure, an open-source software used to collect, process, and store data from various blockchain applications, the majority are servers from third parties. We note that we excluded the server ‘coins.llama.fi’, as it is likely associated with DeFiLlama itself and used for internal operations.

Error	Count
Block height	53
Asynchronous calls failed	17
Key required	5
Missing field/parameter	5
Undefined/null object	4
Call method failed	3
Invalid token/balance	2
GraphQL error	1

TABLE III: **Errors occurred during TVL computation.** Most of them are related to technical issues when running the TVL-computing code.

C. Heterogeneity of on-chain interactions

We now discuss the network analysis results for the frequency of occurrence in absolute terms of the functions called, of the number of protocols calling each specific function, and for the number of token calls in *balanceOf* functions. Following established methodologies [48], [49], we estimate the parameters $\hat{\theta} = (\hat{k}_{min}, \hat{\alpha})$ and conduct a goodness-of-fit test via a bootstrapping procedure ($N = 1,000$). The resulting p-value indicates if the power law is a plausible fit for the empirical data (i.e., $p \geq 0.1$) or not. A log-likelihood ratio ($\mathcal{R}$) test is conducted to compare the power-law fit against other heavy-tailed distributions (exponential, lognormal, and weibull). While the bootstrap analysis shows that the hypothesis that a power-law distribution is a good fit holds only for the distribution of the number of protocols calling each specific function, in all three cases the power law is either a better fit with respect to the other distributions, or the test is inconclusive. An inflection point in the distribution of the number of token calls in *balanceOf* functions, between the values 400 and 1,000 of the x-axis, indicates that values on the right of the elbow are overrepresented and potentially the existence of a transition region.

Next, we comment the approach used to identify alternative functions likely used to compute TVL. Table V reports the most called on-chain functions in absolute terms, rather than

Server	Count	Server	Count
api.thegraph.com	29	config.rampdefi.com	1
raw.githubusercontent.com	5	analytics.back.popsicle.finance	1
sushi-analytics.onrender.com	3	api.affinedefi.com	1
rpc.ankr.com	3	bridge.orbitchain.io	1
vault-content-api.teahouse.finance	2	api.angle.money	1
tv1-adapter-cache.s3.eu-central-1.amazonaws.com	2	api.axelarscan.io	1
api.myso.finance	2	api.beefy.finance	1
crucible.alchemist.wtf	1	api.clipper.exchange	1
data.cian.app	1	api.cream.finance	1
devapi.ease.org	1	api.daomaker.com	1
knit-admin.herokuapp.com	1	api.debridge.finance	1
explorer.poly.network	1	api.defiedge.io	1
f8wgg18t1h.execute-api.us-west-1.amazonaws.com	1	api.exchange.coinbase.com	1
files.insurace.io	1	api.flashstake.io	1
gateway-arbitrum.network.thegraph.com	1	api.flokifi.com	1
counterstake.org	1	api.goldskey.com	1
graph-node.mainnet.termfinance.io	1	api.hord.app	1
graph-proxy.nftx.xyz	1	api.hotcross.com	1
homora-api.alpha.finance.io	1	api.mean.finance	1
messina.one	1	api.multibit.exchange	1
lsd-subgraph.joinstakehouse.com	1	api.nodes-brewlabs.info	1
bsc-dataseed1.defibit.io	1	api.resonate.finance	1
metabase.internal-streamflow.com	1	api.staking.ankr.com	1
midgard.ninerealms.com	1	api.studio.thegraph.com	1
moonbeam.public.blastapi.io	1	api.tokensfarm.com	1
partner-api.stafi.io	1	api.unrekt.net	1
polygon-rpc.com	1	api.vesper.finance	1
preserver.mytokenpocket.vip	1	app.everrise.com	1
stakehouse-subgraph.joinstakehouse.com	1	assets.nabox.io	1
static.optimism.io	1	backend.mochi.fi	1
token-list.solv.finance	1	beaconcha.in	1
universe.staderlabs.com	1	bsc-dataseed.binance.org	1

TABLE IV: **External servers utilized in TVL computations.** An occurrence is counted each time a protocol interacts with a server. While some pose a smaller threat to verifiability, e.g., TheGraph, others appear to be associated with specific protocols.

Method	Count	N of protocols	Method	Count	N of protocols
balanceOf	102655	641	underlying	1613	43
getLockedTokenAtIndex	44022	1	escrows	1203	1
balanceOfUnderlying	4191	3	token0	1040	94
getCurrentTokens	3436	10	token1	1040	94
getReserves	3016	121	tokenByIndex	875	2
totalSupply	2577	154	balance	874	9
token	2456	24	get_coins	814	2
symbol	1835	64	pool_list	809	1
eth_getBalance	1696	170	getEthBalance	791	7
poolInfo	1672	25	calcTotalValue	773	1

TABLE V: **Most called functions in absolute terms.** A number of functions (e.g., *balanceOfUnderlying*, *balance*) have names indicating that they are likely used to compute balance in a non-standard way.

being ranked by the number of protocols that call them. We notice that a number of function names (*balanceOfUnderlying*, *balance*, as well as *totalAssets* that appeared in Table I), indicate functions that are likely to compute balance in alternative ways with respect to the *balanceOf* most common method. To identify all these functions, we use a set of regular expressions that capture any method whose function name includes the term ‘balance’, or a case-insensitive combination of the following terms: ‘total, get, locked’ AND ‘tv1, Ether, ETH, stake, underlying, reserve, amount, supply, value,

locked, shares, asset, liquidity’. Furthermore, we conduct a manual check to remove functions that are clearly not computing TVL but provide supplementary functionalities, such as *totalSupply* or *getReserves*. Table VI reports the full list of alternative functions to *balanceOf* likely used to compute TVL. In total, we remove the following 21 functions: *getAssetInfo*, *getAssetsPrices*, *getAssetsWithState*, *getBNFTAssetList*, *getLiquidityPools*, *getLockedTokenAtIndex*, *getNumLockedTokens*, *getReserveData*, *getReserves*, *getReservesData*, *getReservesList*, *getUnderlyingAsset*, *getUnderlyingOfIBTAddress*,

Function name	Hex Signature	Function name	Hex Signature
accountedBalance	['0x0937eb54']	getTotalRPLStake	['0x9a206c8e']
allBalances	['0x555b6162']	getTotalReserves	['0x242693d3']
balance	['0xb69ef8a8']	getTotalUnderlying	['0xb40494e5']
balanceOf	['0x00fdd58e', '0x35ee5f87', '0x3656eec2', '0xf7888aec']	getTotalValueInPool	['0xc8ecaf30']
balanceOfUnderlying	['0x3af9e669']	getTrackedAssets	['0xc4b97370']
balances	['0x4903b0d1', '0x065a80d8', '0x8909aa3f', '0x27e235e3']	getTvl	['0xd075dd42']
borrowBalanceStored	['0x95dd9193']	getUnderlying	['0x9816f473']
calcTotalValue	['0xc7de38a6']	getUnderlyingBalances	['0x1322d954']
checkBalance	['0x5f515226']	getUnderlyings	['0xf65baefa']
currencyBalance	['0x5c75347a']	getUnderlyingsAmounts-FromClusterAmount	['0x9bb1bebb']
currentTotalStake	['0xce4843e9']	lockedBalances	['0x0483a7f6']
getAllAssets	['0x2acada4d']	lockedLiquidityOf	['0xd9f96e8d']
getAllStakes	['0x04238994']	lockedStakesOf	['0x1e090f01']
getAssets	['0x67e4ac2c']	lockedSupply	['0xca5c7b91']
getBassets	['0x1d3ce398']	poolBalance	['0x96365d44']
getCacheBalances	['0x4a9d1036']	syncBalance	['0xfd9c652b']
getContractValue	['0xdc82697c']	totalAsset	['0xf9557ccb']
getETHPx	['0xab9aadfe']	totalAssetAmount	['0xfd27152c']
getEthBalance	['0x4d2301cc']	totalAssetBorrow	['0x20f6d07c']
getLockedVestings	['0x344e58d3']	totalAssets	['0x01e1d114']
getPoolAmount	['0x945eb764']	totalBalance	['0xad7a672f']
getPoolTotalValue	['0xf1437c16']	totalBalanceOf	['0x4b0ee02a']
getRawFundBalances	['0xe2e4c60c']	totalETH	['0x36bdee74']
getRawFund-BalancesAndPrices	['0x0d8f8a90']	totalReserve	['0x4c68df67']
getReserveTotalBorrows	['0xe6d18190']	totalReserves	['0x8f840ddd']
getSupply	['0xf77ee79d']	totalSYNCLocked	['0xc3f4d79f']
getSupportedAsset	['0x60a8b18a']	totalStaked	['0x817b1cd2']
getSupportedAssets	['0xe5406dbf']	totalTokenBalanceStakers	['0x1878fbf3']
getSupportedAssetsLength	['0xc0fd22b7']	totalValue	['0xd4c3eea0']
getToken1Balance	['0x5153786b']	total_staked	['0xaf7568dd']
getTotalAmounts	['0xc4a7761e']	underlyingBalance	['0x59356c5c']
getTotalAsset	['0x2768385d']	virtualBalance	['0xcdcd2af17']
getTotalBalance	['0x12b58349']	virtualUsdtAccumulatedBalance	['0xd88953b4']
getTotalPooledEther	['0x37cfdaa']	yvCurveFRAXBalace	['0xc645065e']

TABLE VI: **Alternative functions to balanceOf likely used to compute TVL.** For each function we report the name (1st column) and its hex signature (2nd column).

getUnderlyingPrice, *getUnderlyingTokenAddress*, *totalShares*, *totalSupply*, *totalUnderlying*, *totalUnderlyingSupply*.

D. Changes to computation methods over time

The analysis in Section III-D follows the intuition that developers can modify how TVL is computed and that the collection may be dependent on one specific implementation, thus TVL computations might change across time and commits. We thus repeat the data gathering on the following commits:

- 6764756f9270ab6a3047c06c13c0b1b2d32a3247: Jan 4 2024;
- aef637c6413b5101667e567a0422769b1ca99564: Oct 4 2023;
- 1bf7b5c798c7a491694882a7b390ed41385c315: Jul 4 2023;
- 72b764c5d0bb5896f2857dd8c2ced5e89e7fb063: Apr 4 2023;
- 679f52e123a19d9c5160d1aa79a33d9de6dc5ec: Jan 4 2023;

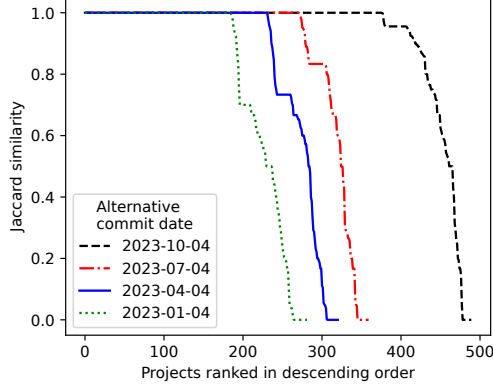
As discussed in the main body of the paper, we do not analyze data earlier than 2023: the structure of some critical files within the DeFiLlama repository has changed in time, thus making our data-gathering infrastructure less reliable on older commits. In particular, the number of comparable commits decreases steadily with time, and for commits older than Aug 2021, our pipeline becomes not compatible with specific changes made to the repository (see <https://github.com/DefiL>

lama/DefiLlama-Adapters/discussions/432). Table VII shows that differences in the dataset of captured calls across commits exist: the total amount of calls to an Ethereum node is not stable over time, and few specific protocols play a relevant role in this sense: as one can see in Table VII, the differences are markedly smaller when some specific projects are excluded.

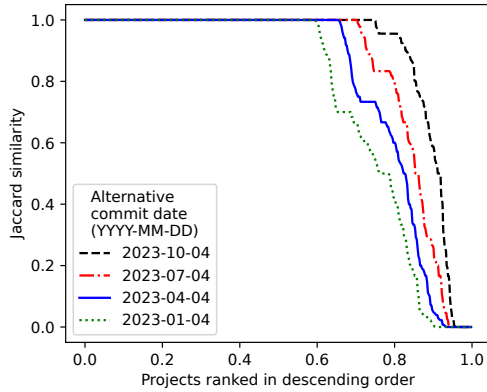
	Jan 24	Oct 23	Jul 23	Apr 23	Jan 23
Total calls	197 775	333 239	262 512	314 280	254 259
Unicrypt calls	44 032	198 228	150 094	104 165	77 063
Other calls	153 743	135 011	112 418	210 115	177 196

TABLE VII: **Dataset dimension in different commits.** One specific protocol (Unicrypt) is responsible for large variations across commits.

Next, in Figure 6 we report additional information on the Jaccard similarity analysis. We recall that the Jaccard similarity coefficient, utilized for measuring the similarity of overlapping sets, is defined as the ratio between their intersection and their union, and ranges between 0 and 1. We therefore favor it over alternative similarity metrics that mea-



(a) Figure 3, x-axis non normalized



(b) Figure 3, all protocols

Fig. 6: Jaccard similarity. Panel (a) reports the same information of Figure 3, but projects are reported on the x-axis in absolute terms instead of being normalized. The sample of comparable protocols decreases sharply when older commits are compared. Panel (b) shows instead the Jaccard similarity values when including also projects that raised errors in the data-gathering process or used external hosts. Average values range from 0.90% to 0.75%.

sure distances between text strings (e.g., Hamming), vectors (e.g., cosine similarity) or that identify correlations across data points (e.g., Spearman or Pearson). The left panel shows the same information of Figure 3, but the x-axis is not normalized. The right panel includes instead all protocols, therefore also the ones that raised errors in the data-gathering process or used external hosts. The average Jaccard values are 0.90%, 0.85%, 0.80%, and 0.75% respectively for Oct, Jul, Apr, and Jan 2023. Notably, we cannot exclude that differences in older commits are due to a less reliable collection. Therefore, our findings can be interpreted as a lower boundary for the similarity values.

Finally, Table VIII reports information on the evolution over time of the ratio between the number of standard and alternative balance queries (excluding protocols that used hosts or raised errors during the data collection).

	Commit identifiers and date				
	676475 (2024-01-04)	aef637 (2023-10-04)	1bfb7b (2023-07-04)	72b764 (2023-04-04)	679f52 (2023-01-04)
Alt. functions	6831	7554	6836	6882	7197
Std. balance queries	70073	54144	27534	23426	18367
Ratio	0.089	0.122	0.199	0.227	0.282

TABLE VIII: Evolution over time of the ratio between the number of standard and alternative balance queries. Standard balance queries include *balanceOf* and *eth_getBalance* calls.

E. Non-mutually exclusive smart contract calls

We report the list of duplicated addresses in Tables IX and X. These correspond to calls executed by different protocols (column ‘Protocols’) on the same input address (column ‘Input’) and token address (only for Table X, column ‘On’). Upon closer inspection, we find that in most cases they are related to interconnected protocols (see, e.g., metis and metisBridge and Curve and Bent). One possible explanation for this phenomenon is that these addresses are reported directly by the project developers; we cannot exclude that some of the smart contracts are incorrectly reported multiple times only for a temporary time span and that they are further removed by the maintainers of the DeFiLlama platform.

Input	Protocols
0x3980c9ed79d2c191a89e02fa3529c60ed6e9c04b	[‘metis’ ‘metisBridge’]
0x8301ae4fc9c624d1d396cbdaa1ed877821d7c511	[‘curve’ ‘bent’]
0xb576491f1e6e5e62f1d8f26062ee822b40b0e0d4	[‘curve’ ‘bent’]
0xd51a44d3fae010294c616388b506acda1bfaae46	[‘curve’ ‘bent’]
0xdc24316b9ae028f1497c275eb9192a3ea0f67022	[‘curve’ ‘bent’]

TABLE IX: Duplicated *get_ethBalance* functions. *get_ethBalance* calls executed by different protocols on the same input address (column ‘Input’).

F. TVL reconstruction from on-chain data

To conduct the analyses in Section F, we collect on-chain data for each protocol-specific address and its associated tokens as follows. We inspect calls that represent balance queries for Ether or tokens, assuming that addresses queried during computation contribute to the project TVL. We obtain a list of addresses contributing towards the locked value for each project, along with a compilation of tokens ($N = 12246$) in which all projects hold value. Next, for each protocol, we extract cryptoassets balances and prices. To obtain quantities, we use the lists of protocol-specific addresses and tokens to query their state and extract historical balance information monthly from January 1st 2021 to February 1st 2024. To price tokens in a common currency, for each token we search for trading pairs between that token and wrapped Ether (wETH) on the Uniswap V2 DEX until January 2024, extract the logged liquidity changes occurring after any trading or liquidity provision action, and compute their implied exchange rate

Input	On	Protocols	Input	On	Protocols
0x031816fd...	0x03e173ad...	['dodo' 'thales']	0x031816fd...	0xc02aaa39...	['dodo' 'thales']
0x0f41eade...	0xc36442b4...	['pawncfi-lending' 'pawncfi-nft']	0x16770d64...	0xc02aaa39...	['enzyme' 'diva']
0x19b080fe...	0x96e61422...	['keep3r' 'curve']	0x1a26ef65...	0x42bbfa2e...	['bobagateway' 'boba']
0x1a26ef65...	0xd26114cd...	['bobagateway' 'boba']	0x1ce8aafb...	0xae7ab965...	['enzyme' 'diva']
0x23012599...	0xba30e5f9...	['pawncfi-lending' 'pawncfi-nft']	0x25d6fe0d...	0x49cf6f5d...	['pawncfi-lending' 'pawncfi-nft']
0x27e49962...	0x790b2cf2...	['pawncfi-lending' 'pawncfi-nft']	0x27f23c71...	0xc02aaa39...	['enzyme' 'nexus']
0x306b1950...	0xeca82185...	['uma' 'perlinx']	0x325a0e5c...	0xae7ab965...	['swell-vault' 'enzyme']
0x325a0e5c...	0xc02aaa39...	['swell-vault' 'enzyme']	0x32ecc1de...	0xb7f7f6c5...	['pawncfi-lending' 'pawncfi-nft']
0x3980c9ed...	0x1f9840a8...	['metis' 'metisBridge']	0x3980c9ed...	0x2260fac5...	['metis' 'metisBridge']
0x3980c9ed...	0x3405a1bd...	['metis' 'metisBridge']	0x3980c9ed...	0x4fab145...	['metis' 'metisBridge']
0x3980c9ed...	0x51491077...	['metis' 'metisBridge']	0x3980c9ed...	0x6226e00b...	['metis' 'metisBridge']
0x3980c9ed...	0x6b175474...	['metis' 'metisBridge']	0x3980c9ed...	0x6b359506...	['metis' 'metisBridge']
0x3980c9ed...	0x7fc66500...	['metis' 'metisBridge']	0x3980c9ed...	0x9e32b13c...	['metis' 'metisBridge']
0x3980c9ed...	0xa0b86991...	['metis' 'metisBridge']	0x3980c9ed...	0xba6b0dbb...	['metis' 'metisBridge']
0x3980c9ed...	0xd533a949...	['metis' 'metisBridge']	0x3980c9ed...	0xdac17f95...	['metis' 'metisBridge']
0x3a93e863...	0xeca82185...	['uma' 'perlinx']	0x3e75dcad...	0xa0b86991...	['domfi' 'uma']
0x3f1b0278...	0xfafdf0c4...	['keep3r' 'curve']	0x41284a88...	0x0bc529c0...	['percent' 'balancer-v1']
0x41284a88...	0xc02aaa39...	['percent' 'balancer-v1']	0x43b4dfdd...	0xabc6da0f...	['curve' 'bent']
0x46f5e363...	0xeca82185...	['uma' 'perlinx']	0x4a2f0ca5...	0x1f573d6f...	['bancor' 'ichifarm']
0x4a2f0ca5...	0x903bef17...	['bancor' 'ichifarm']	0x4e8d60a7...	0xc02aaa39...	['degenerative' 'uma']
0x4f1424ce...	0xa0b86991...	['degenerative' 'uma']	0x505efcc1...	0x04abeda2...	['nest' 'parasset']
0x516f5959...	0xc02aaa39...	['degenerative' 'uma']	0x55a8a39b...	0x99d8a9c4...	['curve' 'bent']
0x55a8a39b...	0xa47cb8f3...	['curve' 'bent']	0x58378f5f...	0x903bef17...	['ichifarm' 'balancer-v1']
0x58378f5f...	0xc02aaa39...	['ichifarm' 'balancer-v1']	0x5a6a4d54...	0x99d8a9c4...	['curve' 'bent']
0x5eeaf7d...	0xed5af388...	['pawncfi-lending' 'pawncfi-nft']	0x5f0a4a59...	0xabc4ca0ed...	['pawncfi-lending' 'pawncfi-nft']
0x7514799c...	0xe012ba8f...	['pawncfi-lending' 'pawncfi-nft']	0x799c9518...	0xa0b86991...	['degenerative' 'uma']
0x7c62e5c3...	0xc02aaa39...	['degenerative' 'uma']	0x7d0b6fb1...	0x60e4d786...	['pawncfi-lending' 'pawncfi-nft']
0x82c427ad...	0xc02aaa39...	['opyn-squeeth' 'uniswap']	0x8301ae4f...	0xc02aaa39...	['curve' 'bent']
0x8301ae4f...	0xd533a949...	['curve' 'bent']	0x8461a004...	0x95dfdc81...	['keep3r' 'curve']
0x8818a9bb...	0x555f75e...	['keep3r' 'curve']	0x94e653af...	0xa0b86991...	['domfi' 'uma']
0x99e58237...	0x6b175474...	['percent' 'balancer-v1']	0x99e58237...	0xc02aaa39...	['percent' 'balancer-v1']
0x9a5c88ac...	0x04abeda2...	['nest' 'parasset']	0x9c2c8910...	0x1cc481ce...	['keep3r' 'curve']
0x9d046499...	0x62b9c735...	['curve' 'bent']	0x9d046499...	0xd533a949...	['curve' 'bent']
0x9fe9bb6b...	0x9ea3b5b4...	['delta' 'core']	0xaa5a67c2...	0x86537736...	['curve' 'bent']
0xaff95ac1...	0x903bef17...	['rari' 'ichifarm']	0xb1a3e5a8...	0xa0b86991...	['degenerative' 'uma']
0xb39dbcb5...	0xe0a97733...	['tokensfarm' 'bloxmove']	0xb40ba947...	0xeca82185...	['uma' 'perlinx']
0xb576491f...	0x4e3fbd56...	['curve' 'bent']	0xb576491f...	0xc02aaa39...	['curve' 'bent']
0xba3436fd...	0x1a7e4e63...	['angle' 'curve']	0xba3436fd...	0x1abaea1f...	['angle' 'curve']
0xbaaa1f5d...	0x853d955a...	['curve' 'bent']	0xbaaa1f5d...	0x956f47f5...	['curve' 'bent']
0xbaaa1f5d...	0xabc6da0f...	['curve' 'bent']	0xbebc4478...	0x6b175474...	['curve' 'bent']
0xbebc4478...	0xa0b86991...	['curve' 'bent']	0xbebc4478...	0xdac17f95...	['curve' 'bent']
0xc3160c5c...	0x40803cea...	['spool-v2' 'spool']	0xc48b8329...	0xc00e94cb...	['percent' 'balancer-v1']
0xc48b8329...	0xc02aaa39...	['percent' 'balancer-v1']	0xc697051d...	0x7fc66500...	['aave' 'balancer-v1']
0xc697051d...	0xc02aaa39...	['aave' 'balancer-v1']	0xca2531b9...	0xc02aaa39...	['degenerative' 'uma']
0xceaf7747...	0xa693b19d...	['curve' 'bent']	0xd3a0e00f...	0xa0b86991...	['domfi' 'uma']
0xd50fbace...	0xeca82185...	['uma' 'perlinx']	0xd51a44d3...	0x2260fac5...	['curve' 'bent']
0xd51a44d3...	0xc02aaa39...	['curve' 'bent']	0xd51a44d3...	0xdac17f95...	['curve' 'bent']
0xd632f226...	0x6c3f90f0...	['fraxfinance' 'bent']	0xd632f226...	0x853d955a...	['curve' 'bent']
0xd6ac1cb9...	0x69681f8f...	['keep3r' 'curve']	0xdc24316b...	0xae7ab965...	['curve' 'bent']
0xdcef968d...	0xa0b86991...	['curve' 'fraxfinance']	0xe010fcd4...	0x51491077...	['percent' 'balancer-v1']
0xe010fcd4...	0xc02aaa39...	['percent' 'balancer-v1']	0xe867be95...	0xbal00000...	['percent' 'balancer-v1']
0xe867be95...	0xc02aaa39...	['percent' 'balancer-v1']	0xe969991c...	0xa0b86991...	['percent' 'balancer-v1']
0xe969991c...	0xc02aaa39...	['percent' 'balancer-v1']	0xeb85b2e1...	0xabc16da9d...	['percent' 'balancer-v1']
0xeb85b2e1...	0xc02aaa39...	['percent' 'balancer-v1']	0xee9a6009...	0x2260fac5...	['percent' 'balancer-v1']
0xee9a6009...	0xc02aaa39...	['percent' 'balancer-v1']	0xf083fba9...	0x4e3fbd56...	['curve' 'bent']
0xf083fba9...	0x9e0441e0...	['curve' 'bent']	0xf35a80e4...	0xc02aaa39...	['degenerative' 'uma']
0xf861483f...	0x853d955a...	['fpi' 'curve']	0xf8ef02c1...	0xc02aaa39...	['degenerative' 'uma']
0xfcf434e...	0x1498bd57...	['delta' 'core']	NaN	NaN	NaN

TABLE X: **Duplicated balanceOf functions.** BalanceOf calls executed by different protocols on the same token address (column 'On') and input address (column 'Input').

determined by the liquidity ratio as in [40]. We exclude tokens with very low liquidity and remove price outliers (more details below); in total, using this approach, we could price 942 tokens. For comparisons with USD, we gather historical data from Coingecko on the ETH/USD exchange rate.

Finally, we categorize tokens following the conceptualization of Section II. We extract data from Coingecko and Coinmarketcap [50], [51] to identify stablecoins and governance tokens, and complement this with a semi-automated search of derivative tokens and additional governance tokens based on regular expressions on their names and symbols. We

divide tokens into seven different categories: Ether and its wrapped token wETH, wrapped BTC (wBTC), non-crypto-backed stablecoins, crypto-backed stablecoins, governance tokens, derivative tokens, and others (non categorized) tokens. Specifically, we conduct the following steps:

- **Stablecoins:** we extract information from Coingecko [50] and Coinmarketcap [51] on $N = 64$ stablecoins. We select those with more than 20 mln\$ market capitalization, whose price is stable over time, and pegged to the US dollar (stablecoins pegged to EUR or commodities like gold clearly represent a minor market). Next, we look

into their documentation and distinguish those that are decentralized and backed by other cryptoassets⁴ ($N = 11$) and those that are issued by trusted entities after a deposit of external assets such as cash, treasury bills, or other cash equivalents⁵ ($N = 9$).

- **Governance tokens:** we exploit again Coingecko and Coinmarketcap to extract a list of $N = 110$ governance tokens; we complement it with a regular-expression-based search on the dataset of 12 246 tokens being called in DeFiLlama to compute protocols balances and label additional tokens that contain the word ‘governance’ in their name, for a total of $N = 162$ tokens;
- **Bitcoin tokenized representations through bridges:** whilst other tokens in addition to wrapped Bitcoin (wBTC) exist, wBTC is by far the largest one in the ecosystem. For instance, renBTC and Houobi BTC, two of the main alternatives to wBTC, have a market capitalization of around 20 mln\$ against almost 9 bn\$ of wBTC at the time of writing. Following a similar line of thought, we only focus on Bitcoin, and do not investigate other distributed ledgers and their native tokens.
- **Derivative tokens:** we utilize the dataset of 12 246 tokens called in DeFiLlama to identify them. Upon a deep inspection of the data, we found that token names follow specific patterns; for instance, DEX LP token names typically contain terms like ‘LP’ or ‘Pool’; interest-bearing tokens from protocols like Compound and Aave produce cTokens and aTokens that represent a claim on the underlying asset; similarly, sTokens are receipt tokens for the Synthetix protocol. We devise a procedure based on regular expression searches executed on the token names. We identify a total of 2222 derivative tokens.

We now provide additional details on the procedure envisaged to obtain token prices. As the price time series extracted are noisy, we exclude the tokens having less than 10 000 points, i.e. those that are traded rarely and have very low liquidity, assuming that they do not play a relevant role in the ecosystem. Next, we implement a cleaning procedure to remove from each price time series the outliers, i.e., individual price values that diverge by several orders of magnitude from the trend. To remove them, we compute the centered rolling average on a time window of ten days and remove all values where the difference between the rolling average and the price time series is larger than 20%. While the approach is relatively simple, it provides satisfying results for most tokens, but it is not effective on tokens with low liquidity. We then conduct manual sanity checks to ensure that prices are cleaned correctly and additionally remove the prices of 80 tokens whose values have exceedingly high volatility and therefore it was not possible to determine accurately their correct trend. Future work could investigate these choices more thoroughly and provide an alternative scenario that includes

⁴DAI, Liquity USD, FRAX, Ethena USDe, Decentralized USD, mkUSD, sUSD, Alchemix USD, DOLA, MIM.

⁵USDT, USDC, True USD, BUSD, PAX Dollar, First Digital USD, Paypal USD, Gemini Dollar, Verified USD.

also the removed tokens. However, as for most protocols the value is concentrated in few and widely used tokens, we do not expect findings to change substantially. Finally, we manually check that the stablecoins included in the study did not deviate from their peg and assume that their price is anchored to the US dollar.

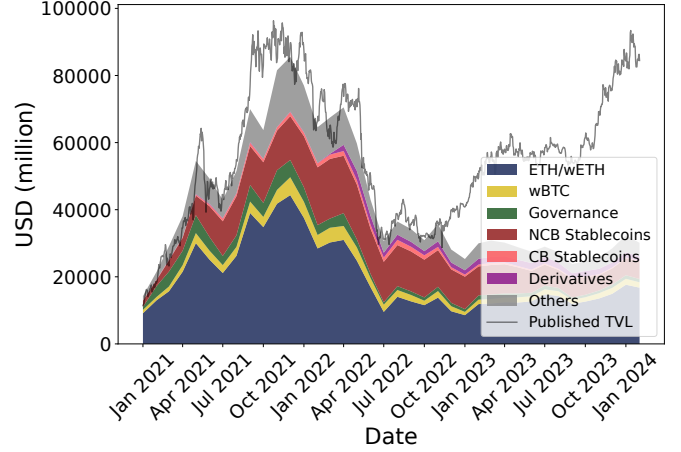


Fig. 7: **TVL reconstruction for the Ethereum DeFi ecosystem.** The plot shows the evolution in time of the vTVL value for all protocols in the case study.

To provide a broader picture of the ecosystem, in addition to the individual protocols described in Section IV-A, we show in Figure 7 the sum of the vTVL of all protocols for which we could recompute values and its evolution in time, again compared to DeFiLlama API data. The off-chain TVL value reported here for comparison is smaller than the TVL of the entire DeFi ecosystem, first because it is computed only on Ethereum, and second because our dataset includes only projects for which we could extract both on-chain and off-chain data. We observe that the vTVL is dominated by Ether and non-crypto-backed stablecoins; the remaining token categories play a less relevant role. It is also possible to notice that the off-chain and on-chain sources correspond only until the end of 2022. However, we argue that in this context the comparison between on-chain data and DeFiLlama API data is for reference and illustrative purpose only. Indeed, we noticed that for some protocols the API data are reported only after a certain date, while we find on-chain assets associated with them also before that date. This inflates the quantity of on-chain assets in comparison with the off-chain ones for earlier dates. Second, our infrastructure did not capture standard balance calls enabling to reconstruct the TVL of the protocol Lido, which is one of the largest protocols in terms of TVL according to DeFiLlama, especially after the end of 2022. Third, by investigating the largest protocols, we found that the API files do not always report data for protocol versions separated consistently with respect to how they are reported in the GitHub DeFiLlama repositories. For instance, Aave APIs report together Aave v2 and v3 (the repository reports one folder for Aave-v1 and one named

‘Aave’ seemingly for v2 and v3). The Uniswap API file reports together v2 and v3; however, Uniswap-v1, -v2, and -v3 are reported in a separate folder (we thus corrected API data accordingly). We also note that Uniswap -v1 and -v2 values are partly computed through the use of external hosts, while for Uniswap v3 we captured on-chain interactions. Similarly, a steadily discrepancy emerges for the protocol Maker at the end of 2022. Also in this case, as explained in the main body of the paper, this could be due to the use of external hosts to compute TVL, but this pattern is also consistent with the possibility that the APIs include both Maker and its related project Maker RWA. All these considerations explains at least in part the structural difference that emerges in the comparison of on-chain and off-chain sources between and after end of 2022. Finally, as the change also corresponds with the Merge (which took place on September 2022), we cannot exclude that also this event has a role. This aspect needs further investigation.

G. TVL Composition Changes Across Categories & Time

In this section we examine TVL composition changes by posing specific attention to the tokens included in the calculation of TVR introduced by Luo et al. [8]. Specifically, we analyze how the ratio between the value of assets used to measure TVR and the value of TVL, $R = \frac{TVR_a}{TVL}$, changes across protocol category, protocol size, and time.

We utilize the API DeFiLlama data to avoid data loss and utilize our token categorization to compute the amount of each protocol as the US dollar amount of plain tokens, i.e., native tokens, governance tokens, and NCB stablecoins. We focus on the subset of protocols ($N = 412$) with average TVL larger than $5 \cdot 10^4$ USD to ensure noise reduction and group them in DeFi categories as reported by DeFiLlama, focusing on Derivatives ($N = 19$), DEXs ($N = 69$), Lending ($N = 46$), Staking ($N = 17$), and Yield ($N = 47$) protocols. The remaining ones (including Collateralized Debt Position or CDP protocols, Services, Real World Assets or RWA, NFT Marketplaces, ...) are categorized as ‘Others’. Figure 8 reports the ratio R for these protocol categories without distinguishing them based on size or time. We find that the median value of R is respectively 90.4%, 68.3%, 55.3%, 94.4%, 23.8%, and 51.4%. Yield protocols highly rely on non-redeemable tokens, while Derivatives and Staking protocols have the highest ratio, indicating lower reliance on them.

Figure 9 shows two panels conveying additional information on the ratio R , with protocols further divided by size (upper panel) and time (lower panel). More precisely, on top we divide protocols in small size ($N = 117$), medium size ($N = 232$), and large size ($N = 63$), respectively when they have average TVL below 10^6 USD, between 10^6 USD and 10^8 USD, and larger than 10^8 USD. On the bottom, we split the R values for each protocol across time (2021, 2022, 2023, 2024). Interestingly, we do not find strong patterns when distinguishing protocols by size. Instead, in Panel (b) we observe that for DEXs, Lending, and Derivatives protocols (excluding 2021), the median of R is steadily decreasing over time, indicating increased reliance on non-redeemable tokens.

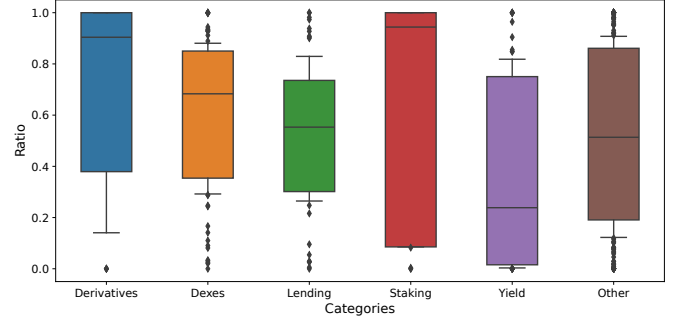


Fig. 8: **Ratio R for different protocol categories.** We compute $R = TVR_a/TVL$ as the ratio between the value of assets used to measure TVR and the value of TVL, and compare it across protocol categories. Derivatives and staking protocols have the highest R values.

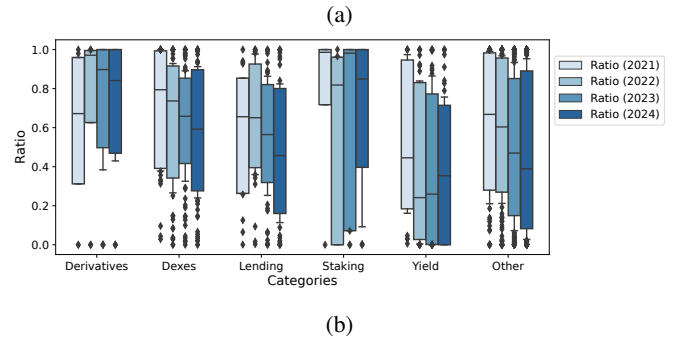
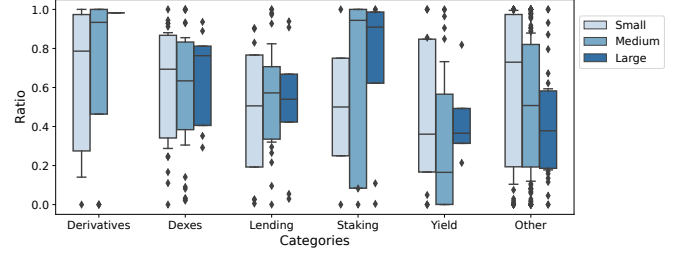


Fig. 9: **Ratio of Total Value Redeemable over Total Value Locked.** Protocols are grouped by category and further divided by size (a) and time (b). The y-axis indicates the ratio R between TVR and TVL. DEXs and Lending protocols rely more on non-redeemable tokens in 2024 w.r.t. earlier years.

Previous volumes in this series

1267 May 2025	Collateralized lending in private credit	Iñaki Aldasoro and Sebastian Doerr
1266 May 2025	Exchange Rate Effects on Firm Performance: A NICER Approach	Nuwat Nookhwun, Jettawat Pattararangrong and Phurichai Rungcharoenkitkul
1265 May 2025	DeFying gravity? An empirical analysis of cross-border Bitcoin, Ether and stablecoin flows	Raphael Auer, Ulf Lewrick and Jan Paulick
1264 April 2025	Inflation cycles: evidence from international data	Alberto Americo, Douglas K G Araujo, Johannes Damp, Sjur Nilsen, Daniel Rees, Rafael Schmidt and Christian Schmieder
1263 April 2025	Fragile wholesale deposits, liquidity risk, and banks' maturity transformation	Carola Müller, Matias Ossandon Busch, Miguel Sarmiento and Freddy Pinzon-Puerto
1262 April 2025	The risk sensitivity of global liquidity flows: heterogeneity, evolution and drivers	Stefan Avdjiev, Leonardo Gambacorta, Linda S Goldberg and Stefano Schiaffi
1261 April 2025	ETFs as a disciplinary device	Yuet Chau, Karamfil Todorov and Eyub Yegen
1260 April 2025	Supply chain transmission of climate-related physical risks	Douglas K G Araujo, Fernando Linardi and Luis Vissotto
1259 April 2025	Let's speak the same language: a formally defined model to describe and compare payment system architectures	Kees van Hee, Anneke Kosse, Peter Wierds and Jacob Wijngaard
1258 April 2025	How accurately do consumers report their debts in household surveys?	Carlos Madeira
1257 April 2025	Macroprudential and monetary policy tightening: more than a double whammy?	Markus Behn, Stijn Claessens, Leonardo Gambacorta and Alessio Reghezza
1256 April 2025	The disciplining effect of bank supervision: evidence from SupTech	Hans Degryse, Cédric Huylebroek and Bernardus Van Doornik
1255 April 2025	Affordable housing, unaffordable credit? Concentration and high-cost lending for manufactured homes	Sebastian Doerr and Andreas Fuster

All volumes are available on our website www.bis.org.